Abhishek Raghav

Btech CSE

CU24250156

Experiment No.: 3 Implementation and Analysis of Spatial Filtering Techniques using Low-Pass and High-Pass Filters.

Aim

To implement and analyze spatial domain filtering techniques using low-pass and high-pass filters for image smoothing, noise reduction, edge enhancement, and feature preservation using Python and OpenCV.

Python Implementation

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
# Load image
# Make sure to upload 'image.jpg' to your Colab environment or replace with a valid path
image = cv2.imread("/content/Screenshot (15).png")

if image is None:
    print("Error: Image not found! Please upload 'image.jpg' or provide a correct path.")
    # Use a dummy image if not found to prevent errors for demonstration purposes
    # In a real scenario, you might want to exit or handle this more robustly
    image = np.zeros((200, 200, 3), dtype=np.uint8)
    print("Using a black dummy image for demonstration.")
```

```
# Convert BGR to RGB for displaying with Matplotlib
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# 1. Gaussian Filter
gassian = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# 2. Median Filter
median = cv2.medianBlur(gray, 5)
```

```
# 3. Average / Mean Filter
average = cv2.blur(gray, (5, 5))
```

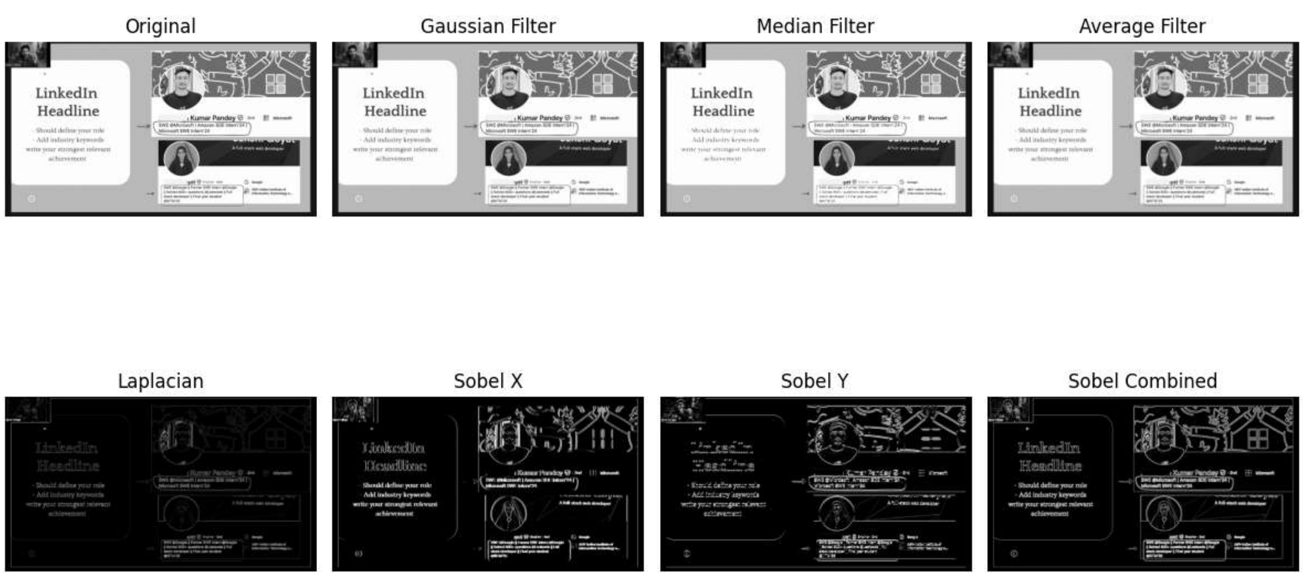
```
# 4. Laplacian Filter
laplacian = cv2.Laplacian(gray, cv2.CV_64F)
laplacian = cv2.convertScaleAbs(laplacian)
```

```
# 5. Sobel Horizontal
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_x = cv2.convertScaleAbs(sobel_x)
```

```
# 6. Sobel Vertical
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
sobel_y = cv2.convertScaleAbs(sobel_y)
```

```
# Display results
plt.figure(figsize=(12, 8))

plt.subplot(2, 4, 1)
plt.imshow(gray, cmap="gray")
plt.title("Original")
plt.axis("off")
```

```
plt.subplot(2, 4, 2)
plt.imshow(gassian, cmap="gray")
plt.title("Gaussian Filter")
plt.axis("off")

plt.subplot(2, 4, 3)
plt.imshow(median, cmap="gray")
plt.title("Median Filter")
plt.axis("off")

plt.subplot(2, 4, 4)
plt.imshow(average, cmap="gray")
plt.title("Average Filter")
plt.axis("off")

plt.subplot(2, 4, 5)
plt.imshow(laplacian, cmap="gray")
plt.title("Laplacian")
plt.axis("off")

plt.subplot(2, 4, 6)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

plt.subplot(2, 4, 7)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

# Combined Sobel
sobel_combined = cv2.magnitude(
    sobel_x.astype(np.float32),
    sobel_y.astype(np.float32)
)

plt.subplot(2, 4, 8)
plt.imshow(sobel_combined, cmap="gray")
plt.title("Sobel Combined")
plt.axis("off")

plt.tight_layout()
plt.show()
```



The experiment specifically asks for Gaussian, Median, Average, Laplacian and Sobel filtering and comparison of their outputs.

Answers to Questions

1. What is spatial filtering? How is it used in digital image processing?

Spatial filtering is a technique in which the value of a pixel is modified according to its neighboring pixels using a filter or convolution kernel.

It is used for:

- Image smoothing
- Noise reduction
- Edge detection
- Sharpening
- Feature enhancement

2. Differentiate between Low-Pass Filters and High-Pass Filters.

Low-Pass Filter	High-Pass Filter
Removes high-frequency components	Emphasizes high-frequency components
Produces a smoother image	Produces sharper details
Reduces noise	Enhances edges
Used for blurring	Used for edge detection
Examples: Gaussian, Average, Median	Examples: Sobel, Laplacian

3. Compare Average, Gaussian and Median Filters.

Filter	Working Principle	Main Application
Average	Replaces pixel with average of neighboring pixels	General smoothing
Gaussian	Uses weighted average based on Gaussian distribution	Noise reduction and smoothing
Median	Replaces pixel with median of neighboring values	Salt-and-pepper noise removal

The experiment asks specifically for comparison of these three filters based on their working principles and applications.

4. Why is Median Filter effective for salt-and-pepper noise?

The Median Filter replaces each pixel with the median value of its neighborhood.

Salt-and-pepper noise creates extremely bright or dark pixels. Since the median is less affected by these extreme values than an average, the noise can be removed while preserving edges relatively well.

5. Explain the role of convolution kernels in spatial filtering.

A convolution kernel is a small matrix of values used to process neighboring pixels.

The kernel is moved across the image and mathematical operations are performed between the kernel values and image pixels.

For example, a simple average kernel is:

```
1/9  1/9  1/9
1/9  1/9  1/9
1/9  1/9  1/9
```

It averages neighboring pixels and produces a smoothing effect.

6. What is the purpose of Sobel and Laplacian operators?

Sobel operator: Detects edges by calculating image intensity gradients in horizontal and vertical directions.

Laplacian operator: Detects rapid changes in intensity using second-order derivatives and is useful for identifying edges and fine details.

The lab specifically requires Sobel filtering in horizontal and vertical directions and Laplacian filtering for edge enhancement.

7. Why are filtering operations essential preprocessing steps in computer vision?

Filtering improves the quality of input images before further processing.

It can:

- Remove unwanted noise
- Smooth images
- Enhance important features
- Detect edges
- Improve segmentation and object detection

The lab notes that spatial filtering is useful in applications such as medical imaging, surveillance, remote sensing and autonomous systems.

8. Discuss the trade-off between image smoothing and edge preservation.

Strong smoothing reduces more noise but can also remove important edges and fine details.

Weak smoothing preserves edges better but may leave considerable noise.

Therefore, the filter and kernel size should be selected according to the application.

Example: Median filtering can remove salt-and-pepper noise while generally preserving edges better than simple averaging.

9. Mention four real-world applications of spatial filtering.

Four applications are:

- Medical imaging – noise reduction and enhancement of medical scans.
- Surveillance systems – improving image quality and detecting objects.
- Remote sensing – processing satellite and aerial images.
- Autonomous systems – edge and feature detection for navigation.

These application areas are also identified in the provided lab sheet.

10. Compare spatial domain filtering with frequency domain filtering.

Spatial Domain	Frequency Domain
Works directly on image pixels	Works on frequency components
Uses kernels and convolution	Uses transforms such as Fourier Transform
Generally easier to understand and implement	Often more computationally involved
Good for local operations	Useful for frequency-based noise removal and filtering
Examples: Gaussian, Median, Sobel	Examples: Low-pass and high-pass Fourier filtering